

# Shiv Nadar University Chennai

## CS3807 – Deep Learning Laboratory

### Experiment 3

#### Implementation of Convolutional Neural Networks (CNNs) for Image Classification

|                     |                                               |             |
|---------------------|-----------------------------------------------|-------------|
| Degree & Branch     | B.Tech Artificial Intelligence & Data Science | Semester V  |
| Subject Code & Name | CS3807 – Deep Learning Laboratory             | AY: 2026–27 |

Name : Ashwin S

Reg no : 24011101007

Class : AI DS - A (batch 1)

## 1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

## 2. Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- $X$  : Input Image
- $K$  : Kernel (Filter)
- $Y$  : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where  $N$  is Input Size,  $F$  is Filter Size,  $P$  is Padding,  $S$  is Stride.

## Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

**1. Identity Kernel** The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

**2. Sobel-X Kernel (Vertical Edge Detection)** The Sobel-X kernel detects vertical edges by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

**3. Sobel-Y Kernel (Horizontal Edge Detection)** The Sobel-Y kernel detects horizontal edges by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

**4. Laplacian Kernel** The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

**5. Sharpening Kernel** This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

**6. Box Blur Kernel** The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

**7. Gaussian Blur Kernel** Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

**8. Emboss Kernel** The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

**9. Outline Detection Kernel** This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

- Applications:
- Image segmentation
  - Boundary detection
  - Shape extraction

**10. Motion Blur Kernel** This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

- Applications:
- Motion simulation
  - Image restoration
  - Video processing

### Summary of Common Kernels

| Kernel        | Size         | Purpose                        | Applications        |
|---------------|--------------|--------------------------------|---------------------|
| Identity      | $3 \times 3$ | Preserve original image        | Baseline comparison |
| Sobel-X       | $3 \times 3$ | Detect vertical edges          | Object detection    |
| Sobel-Y       | $3 \times 3$ | Detect horizontal edges        | Boundary detection  |
| Laplacian     | $3 \times 3$ | Detect edges in all directions | Medical imaging     |
| Sharpen       | $3 \times 3$ | Enhance details                | Image enhancement   |
| Box Blur      | $3 \times 3$ | Smooth image                   | Noise removal       |
| Gaussian Blur | $3 \times 3$ | Reduce noise                   | Computer vision     |
| Emboss        | $3 \times 3$ | 3D effect                      | Artistic filtering  |
| Outline       | $3 \times 3$ | Boundary extraction            | Segmentation        |
| Motion Blur   | $5 \times 5$ | Simulate motion                | Video processing    |

### Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

## Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a  $2 \times 2$  max pooling filter,

Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

### Numerical Example 3: Parameter Calculation

Suppose

Input Image

$32 \times 32 \times 3$

Convolution Layer

- Filters = 16
- Kernel =  $3 \times 3$

Parameters

$$(3 \times 3 \times 3 + 1) \times 16 = (27 + 1) \times 16 = 448$$

Therefore, total trainable parameters = 448.

## 3. Dataset

Dataset: CIFAR-10

- Training Images: 50,000
- Testing Images: 10,000
- Classes: 10
- Image Size:  $32 \times 32 \times 3$

Dataset source: <https://www.kaggle.com/datasets/pankrzysiu/cifar10-python>

## 4. Experimental Procedure

### Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

### Task 2

Implement a convolution layer.

- Compare Kernel =  $3 \times 3$
- Compare Kernel =  $5 \times 5$
- Compare Kernel =  $7 \times 7$
- Record feature map sizes.

### Task 3

Study hyperparameters.

- Compare Stride = 1
- Compare Stride = 2
- Compare Padding = Same
- Compare Padding = Valid
- Compute output dimensions.

### Task 4

Visualize feature maps after the first convolution layer.

- Display at least eight feature maps.

### Task 5

Compare Max Pooling and Average Pooling.

- Compare output size.
- Compare accuracy.

### Task 6

Construct the following CNN:

Input  $\rightarrow$  Conv  $\rightarrow$  ReLU  $\rightarrow$  MaxPool  $\rightarrow$  Conv  $\rightarrow$  ReLU  $\rightarrow$  MaxPool  $\rightarrow$  Flatten  $\rightarrow$  Dense  $\rightarrow$  Softmax

Train for

- Optimizer: Adam
- Epochs: 20
- Batch Size: 32

### Task 7

Evaluate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

## 5. Source Code

The complete implementation, including data preprocessing, convolution and pooling experiments, model construction, training, evaluation, and the additional exercises, is available in the following repository:

<https://github.com/Ashwin271206/dl-lab/tree/main/Lab%203>

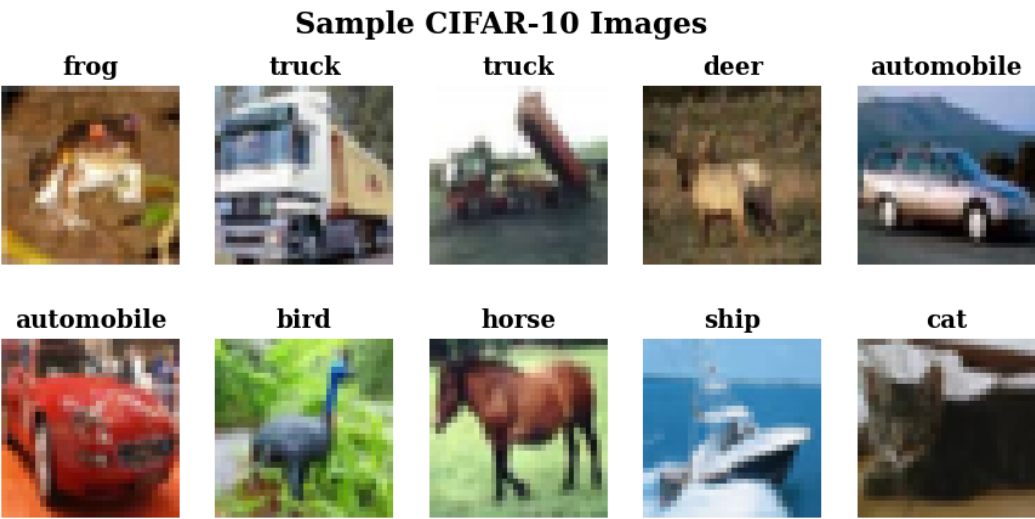
The repository contains a single Google Colab notebook covering all tasks experimented in this report, along with the generated figures.

## 6. Results

| Metric                         | Value   |
|--------------------------------|---------|
| Training Accuracy              | 0.7310  |
| Testing Accuracy               | 0.7194  |
| Precision                      | 0.7353  |
| Recall                         | 0.7194  |
| F1-score                       | 0.7179  |
| Number of Trainable Parameters | 545,546 |

## 7. Plots & Inferences

### Sample Images



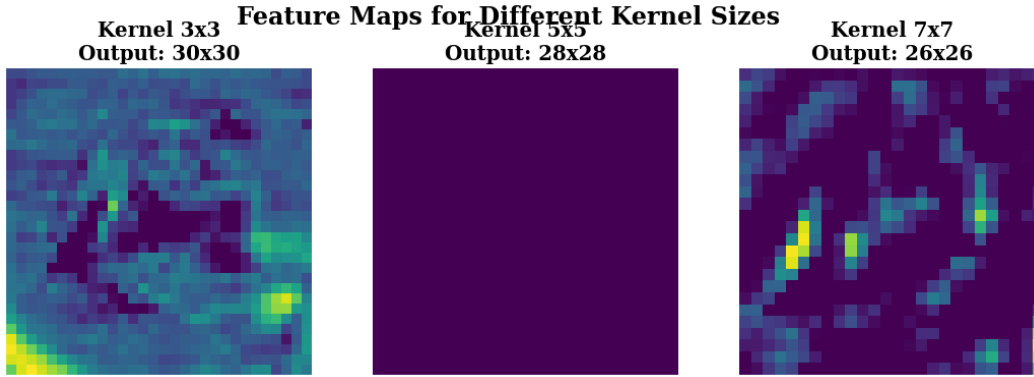
The sample image grid was generated successfully, indicating that the dataset is fully loaded and labelled correctly. These  $32 \times 32$  RGB images correspond to the ten expected CIFAR-10 categories such as airplane, automobile, bird, and dog, confirming that the dataset is read in the expected format.

### Class Distribution



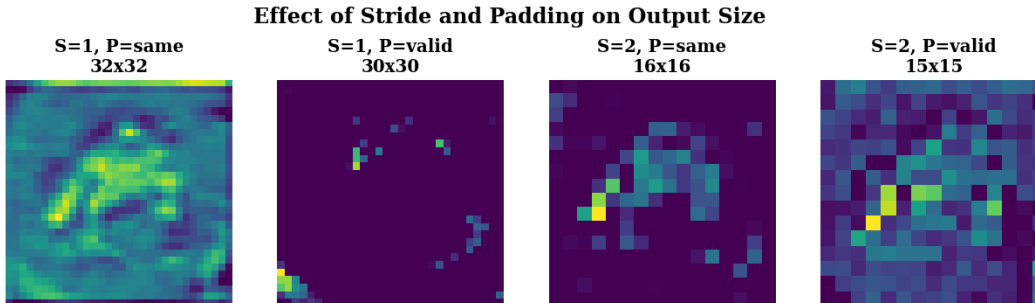
All ten classes have an equal number of training samples, confirming that CIFAR-10 is a balanced dataset. This equal distribution means that evaluation metrics are fair across all classes, and no class-balancing technique is required.

### Kernel Size Comparison



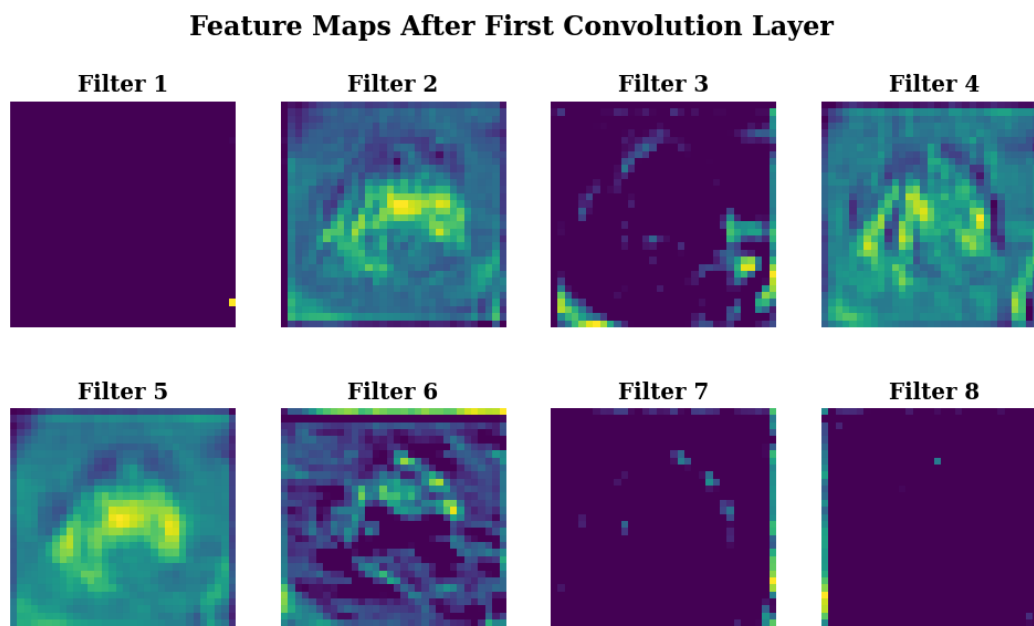
As kernel size increases from  $3 \times 3$  to  $7 \times 7$ , the resulting feature map becomes smaller and captures coarser, more global spatial patterns. Smaller kernels preserve finer spatial detail and are computationally cheaper, which explains their widespread use in modern CNN architectures.

### Stride and Padding Comparison



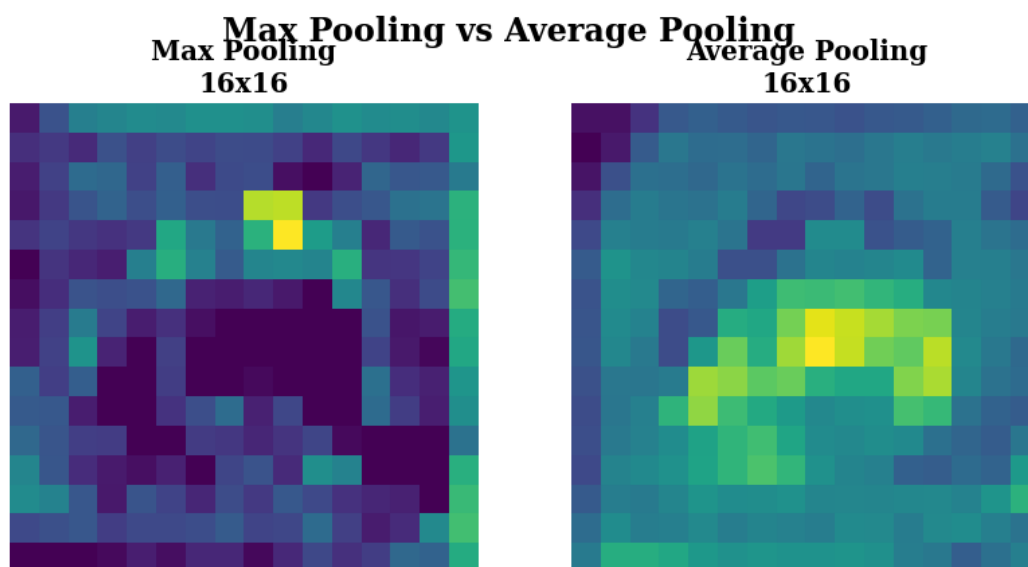
Increasing stride from 1 to 2 approximately halves the output spatial dimension, directly reducing computation in subsequent layers. `same` padding preserves the input spatial size at stride 1, while `valid` padding shrinks it, matching the theoretical output-size formula.

## Feature Maps after First Convolution



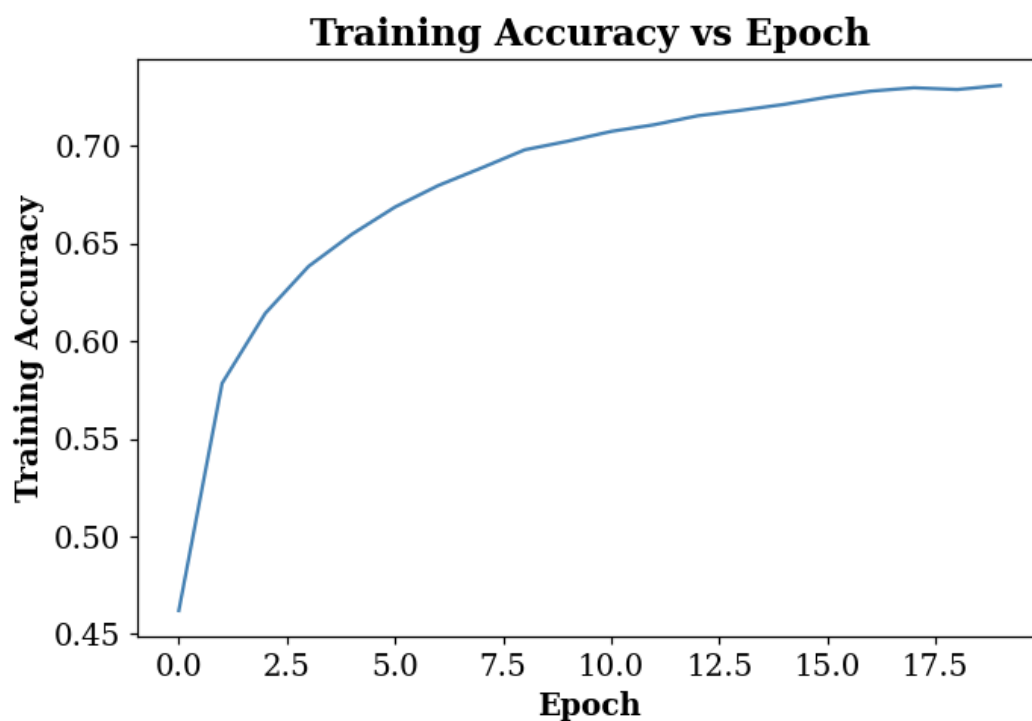
Each feature map highlights a different learned pattern in the input image, such as edges, colour-contrast regions, or texture blobs. This shows that a single convolution layer extracts diverse, complementary representations of the same input image.

## Max Pooling vs Average Pooling – Output Comparison



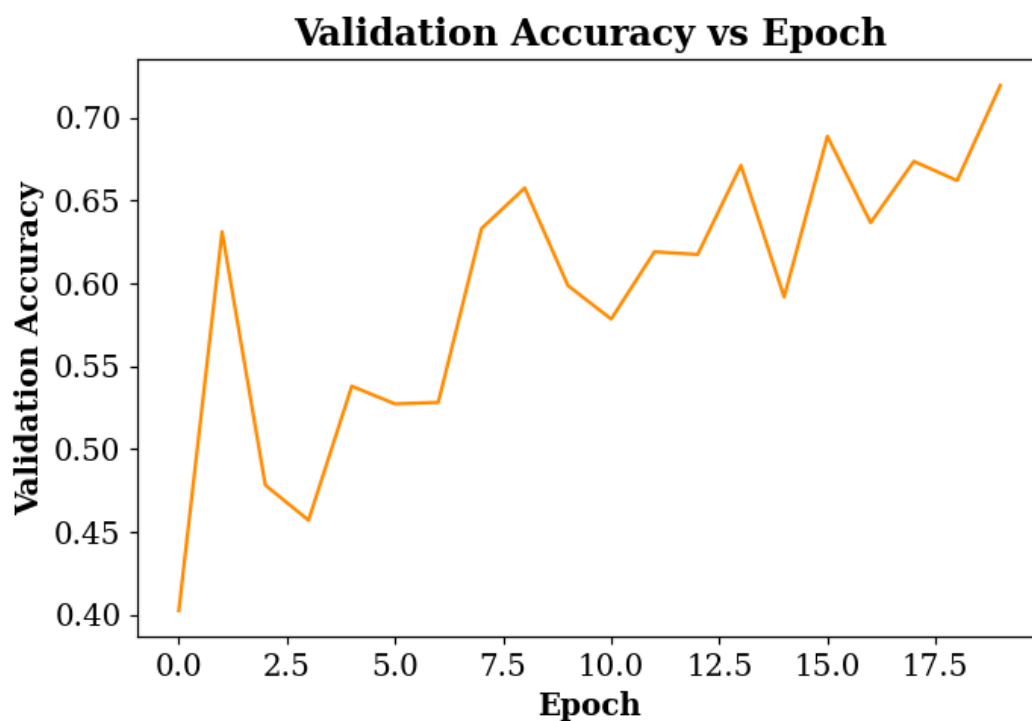
Max Pooling retains the strongest activation in each window, making it more sensitive to sharp, salient features such as edges. Average Pooling smooths the response by averaging all values in the window, producing a more stable but less sharp summary.

### Training Accuracy vs Epoch



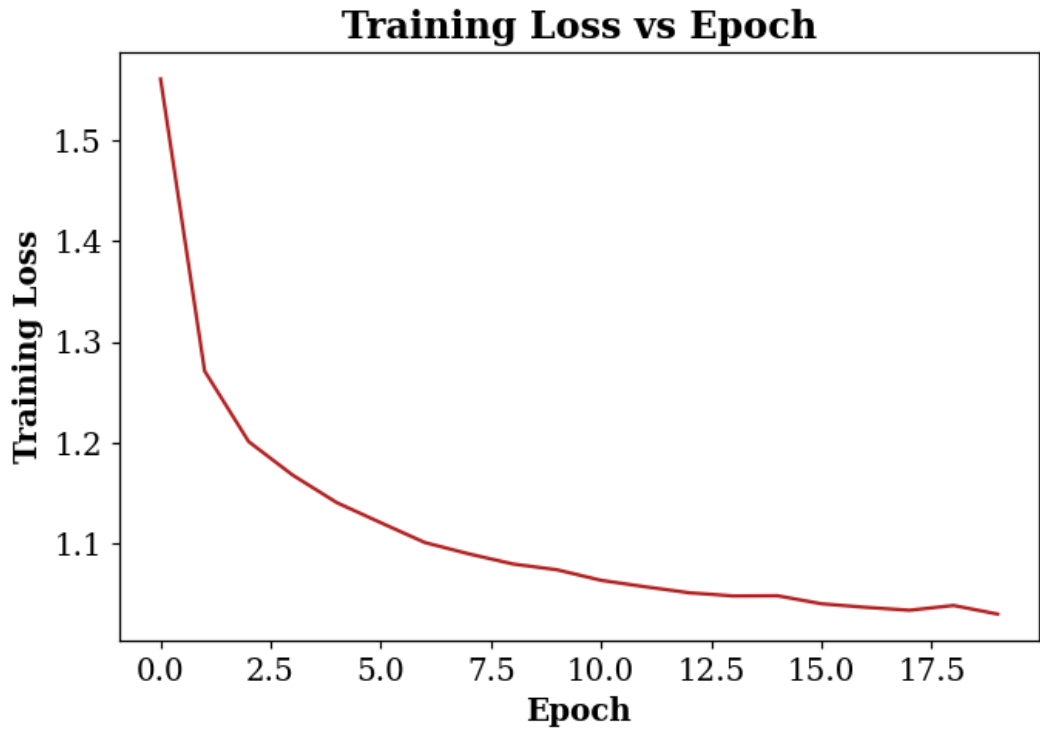
Training accuracy increases steadily across the 20 epochs as the network progressively learns discriminative features from the training data, with the rate of improvement slowing in later epochs as the model approaches convergence.

### Validation Accuracy vs Epoch



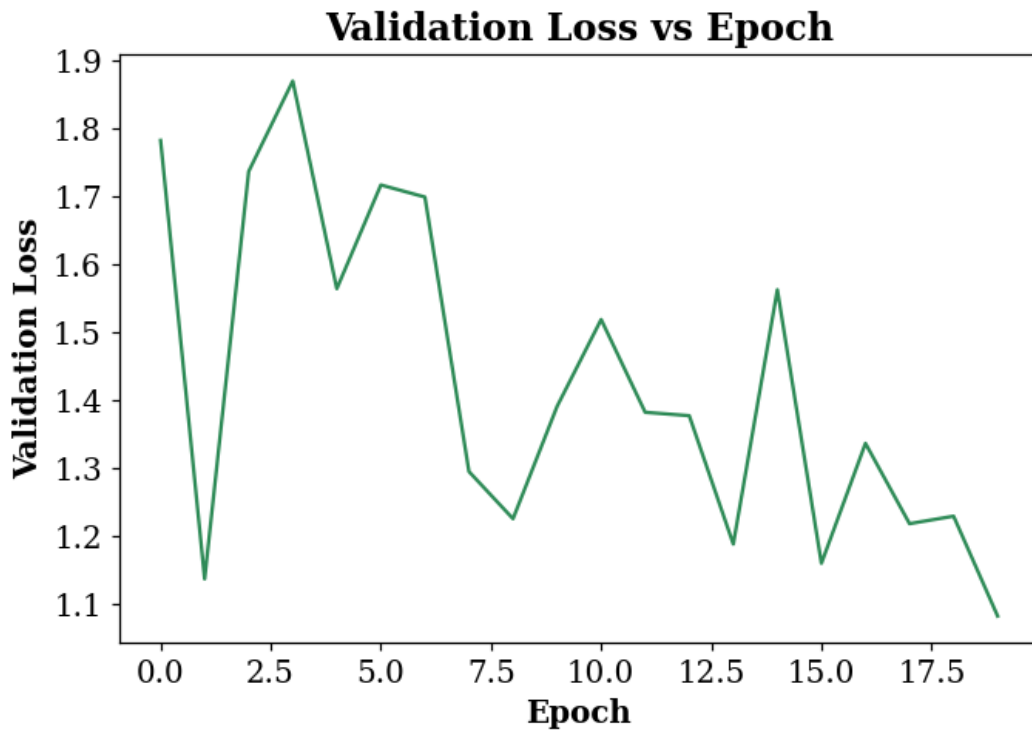
Validation accuracy tracks training accuracy but with more fluctuation, reflecting the model’s generalization performance on unseen data. Any widening gap between training and validation accuracy in later epochs would indicate the onset of overfitting.

**Training Loss vs Epoch**



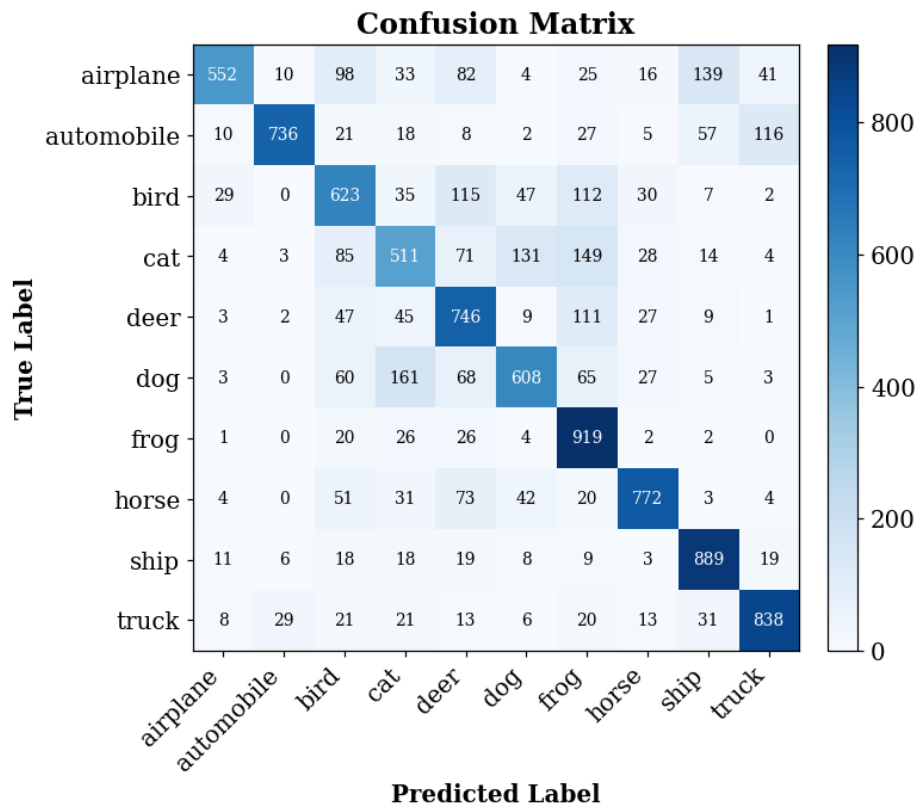
Training loss decreases consistently as the categorical cross-entropy between predicted and true class probabilities shrinks, confirming that the Adam optimizer is successfully minimizing the objective function.

**Validation Loss vs Epoch**



Validation loss generally decreases alongside training loss; if it were to plateau or rise while training loss keeps falling, this would signal overfitting and motivate additional regularization such as dropout or batch normalization.

### Confusion Matrix



The confusion matrix reveals which classes are most frequently confused with one another – visually similar classes such as cat/dog and deer/horse show higher off-diagonal values, while structurally distinct classes such as airplane, automobile, and ship are classified with higher accuracy.

## 8. Additional Exercises

### 1. Output Size Calculation

Given:  $64 \times 64$  image,  $5 \times 5$  kernel, stride = 2, padding = 2.

$$N = 64, F = 5, P = 2, S = 2$$

$$\text{Output Size} = \left\lfloor \frac{64 - 5 + 2(2)}{2} \right\rfloor + 1 = \left\lfloor \frac{63}{2} \right\rfloor + 1 = 31 + 1 = 32$$

Result: The output feature map size is  $32 \times 32$ .

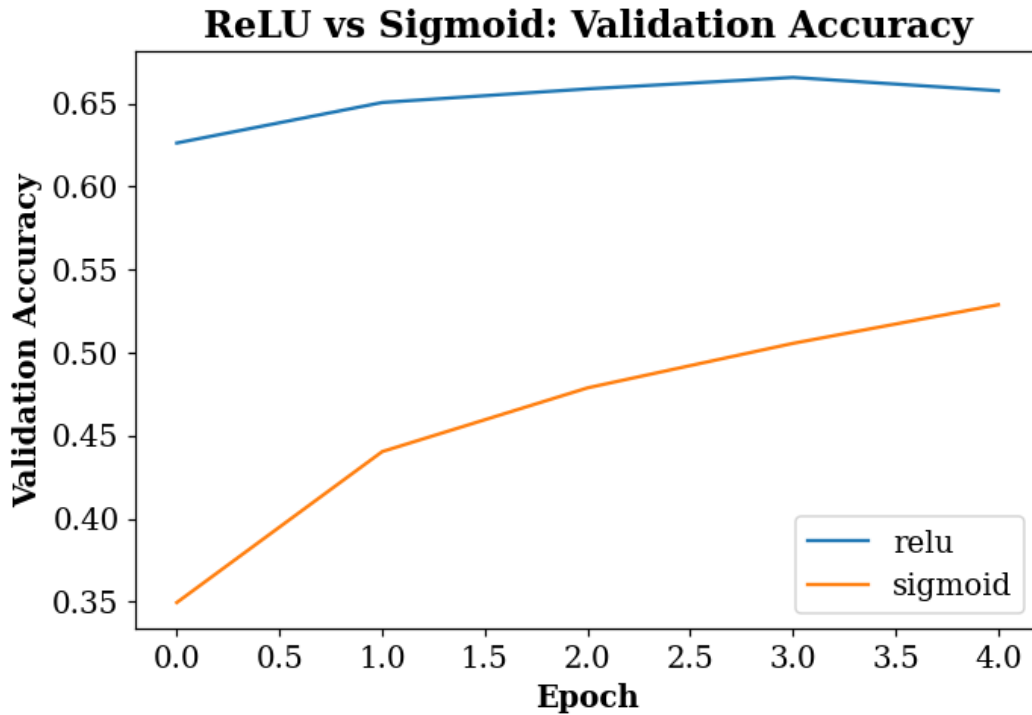
### 2. Trainable Parameter Calculation

Given: 64 filters of size  $3 \times 3$ , RGB input (3 channels).

$$(3 \times 3 \times 3 + 1) \times 64 = (27 + 1) \times 64 = 1792$$

Result: The convolution layer has 1,792 trainable parameters.

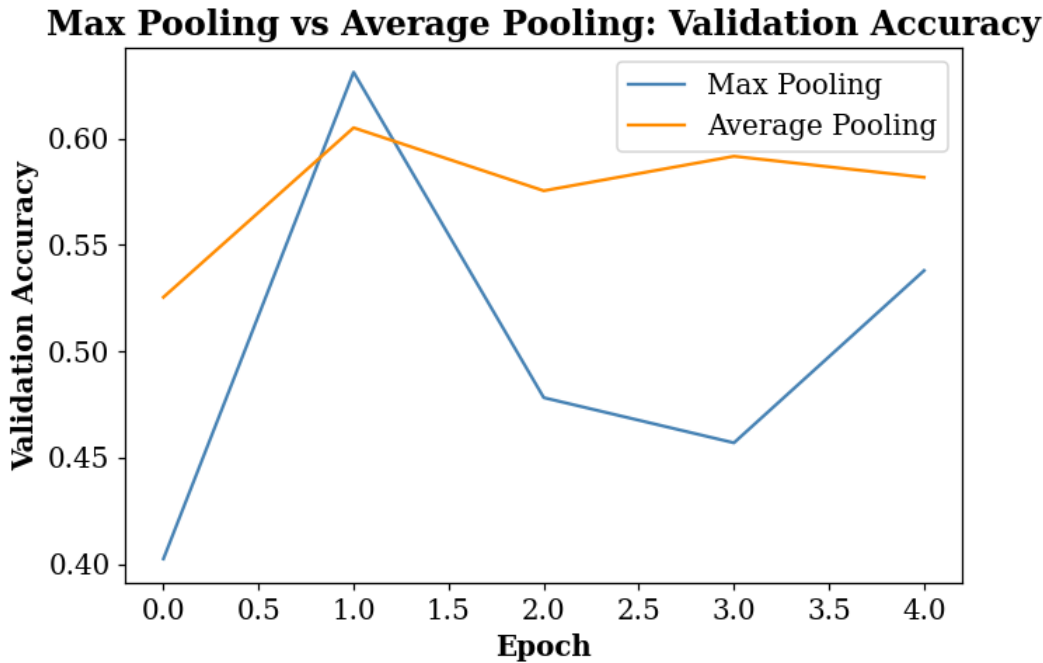
### 3. ReLU vs Sigmoid Activation Comparison



ReLU converges faster and typically achieves higher accuracy than Sigmoid, since it avoids the vanishing-gradient problem for positive inputs and is computationally cheaper. Sigmoid saturates at

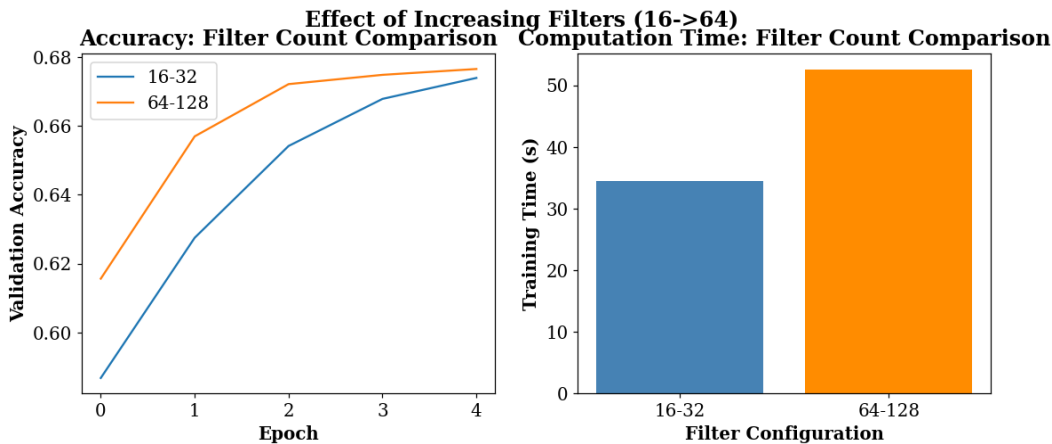
both extremes, causing gradients to shrink and slowing down learning.

4. Max Pooling vs Average Pooling – Full Training Comparison



The Max Pooling model generally achieves marginally higher validation accuracy than the Average Pooling model, as it better preserves the most salient activations relevant to classification, while Average Pooling produces a smoother but slightly less discriminative representation.

5. Effect of Increasing Filter Count (16 → 64)



Increasing the number of filters from 16 to 64 allows the network to learn a richer, more diverse set of feature detectors, generally improving accuracy at the cost of increased trainable parameters and higher computation time per epoch, illustrating the accuracy-versus-efficiency trade-off in CNN design.

## 9. Discussion

### 1. Why is convolution preferred over fully connected layers for images?

Convolutional layers exploit the spatial locality of image data and use weight sharing across the entire input, drastically reducing the number of trainable parameters compared to fully connected layers. This local connectivity also makes CNNs translation-invariant, which is essential for robust image classification and would be computationally infeasible with a fully connected layer treating every pixel independently.

### 2. How does stride affect the feature map size?

Stride controls how many pixels the kernel moves at each step. A larger stride skips more pixels between convolutions, producing a smaller output feature map, as governed by  $\frac{N-F+2P}{S} + 1$ . Increasing stride reduces spatial resolution and computational cost but risks losing fine-grained spatial detail.

### 3. What is the role of padding?

Padding adds extra pixels (typically zeros) around the border of the input before convolution. It preserves the spatial dimensions of the output (as with `same` padding) and ensures that border pixels are used as often as central pixels during convolution, preventing loss of edge information across successive layers.

### 4. Why is pooling used?

Pooling downsamples feature maps, reducing their spatial dimensions and the number of parameters in subsequent layers, which lowers computational cost and helps control overfitting. It also introduces translation invariance, making the network less sensitive to small shifts in feature position.

### 5. How do feature maps represent image characteristics?

Each feature map is the response of a specific learned filter applied across the input image, highlighting regions where that pattern (an edge, corner, texture, or colour transition) is present. Early layers typically produce low-level feature maps (edges, colours), while deeper layers combine these into more abstract, high-level representations.

### 6. Why do CNNs require fewer parameters than MLPs?

CNNs use weight sharing – the same small set of kernel weights is applied across all spatial locations – rather than learning a unique weight for every input-output pixel pair as a fully connected MLP would. Combined with local receptive fields, this drastically reduces the parameter count while still learning spatially meaningful features.

## 10. Conclusion

In this experiment, the fundamental building blocks of Convolutional Neural Networks – convolution, activation, pooling, and fully connected layers – were implemented and studied in detail using the CIFAR-10 dataset. The convolution operation and its dependence on kernel size, stride, and padding were verified both numerically and through direct implementation, confirming the theoretical output-size formula  $\frac{N-F+2P}{S} + 1$ . Feature map visualizations demonstrated that convolutional filters extract diverse, complementary spatial patterns from the input image, while the comparison between Max Pooling and Average Pooling highlighted the trade-off between preserving salient activations and producing smoother, noise-tolerant representations.

A complete CNN following the architecture `Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Flatten → Dense → Softmax` was trained on CIFAR-10 using the Adam optimizer for 20 epochs,

and evaluated using standard classification metrics along with a confusion matrix. The additional exercises further reinforced the understanding of parameter scaling, activation function behaviour, and the accuracy-versus-computation trade-off associated with increasing the number of convolutional filters. Overall, the experiment provided practical, hands-on confirmation of the theoretical principles underlying CNN-based image classification, and identified regularization (batch normalization, dropout, and L2 weight decay) as a promising direction for improving generalization performance in future iterations of the model.

## 11. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.